S.A.I.T.E.

Superior Advanced Intelligence Technological Entity

Relatório técnico

Transformação em IA pessoal local, privada e integrada

Repositório: github.com/Matheus-Fernando-br/S.A.I.T.E

Branch: `claude/new-session-owvmaf` · base auditada: `ec858e81`

Gerado em: 29 de outubro de 2026 às 23:55 (UTC)

★ Atualização de 29/10/2026 — versão na nuvem e sincronizada

> A pedido do dono, o S.A.I.T.E. passou a rodar **na nuvem**, com os mesmos dados em qualquer dispositivo (celular agora; computador, apps, `.exe` e TV depois). Esta seção substitui as partes das seções seguintes que descrevem o backend, o banco e o modelo rodando no computador do usuário.

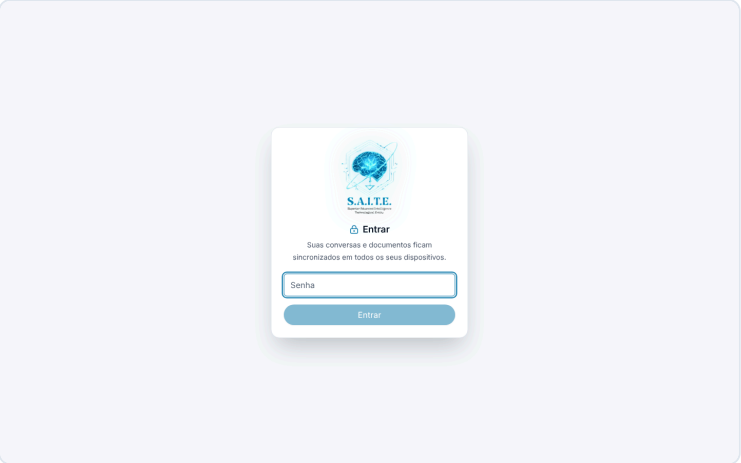
Aspecto	Antes (seções 2–26)	Agora
Backend	FastAPI no computador do usuário	FastAPI no **Render** (serviço `S.A.I.T.E.`, s-a-i-t-e.onrender.com)
Banco	SQLite local + FTS5	**PostgreSQL no Supabase** (projeto `saite`, us-west-1): tsvector em português, embeddings em bytea, arquivos originais no banco; RLS ligado e API pública do Supabase bloqueada
IA	Ollama local (qwen2.5 + bge-m3)	Provedor compatível com a API OpenAI, padrão **Gemini** (gemini-2.5-flash + gemini-embedding-001); Groq/OpenRouter/Ollama por configuração; chat e embeddings separados
Acesso	localhost + token opcional	**Login com a senha do dono**; sessão HMAC por dispositivo (90 dias); trocar a senha revoga tudo; limite contra força bruta
Agente do computador	Processo local consultado pelo backend	Indisponível na nuvem (informado na interface); previsto para o `.exe`

Verificação

Item	Resultado
Backend `pytest` contra PostgreSQL real	**88 aprovados**
Agente local	**17 aprovados**
Ponta a ponta com stack igual à produção (PostgreSQL, senha, provedor OpenAI-compatível)	**15/15 — inclui login, senha errada, sincronização entre dois dispositivos e logout**
Avaliação RAG no PostgreSQL (qwen2.5:1.5b + bge-m3 via API OpenAI-compatível)	**recuperação 100%, fidelidade 100% — igual à versão SQLite**
Esquema aplicado no Supabase	**10 tabelas, RLS em todas, `anon` sem acesso (consultado no projeto real)**

Pendências para entrar no ar

1. Cadastrar no Render os segredos `DATABASE_URL` (Supabase → Connect → Session pooler), `SAITE_LLM_API_KEY` e `SAITE_ACCESS_PASSWORD` (as variáveis não secretas já foram cadastradas).
2. Publicar o código na `main` (o Render e a Vercel publicam a partir dela).
3. Com a chave real, calibrar o limiar de similaridade para `gemini-embedding-001` (o valor 0,45 foi calibrado para bge-m3).



Tela de login (a mesma em todos os dispositivos).

2 Resumo executivo

O S.A.I.T.E. era uma interface de chat (Next.js) ligada a um backend FastAPI que dependia da API comercial da OpenAI e de um banco Supabase remoto. O backend **não iniciava** a partir de uma instalação limpa (pacote `openai` ausente), não havia testes, e várias áreas da interface eram apenas visuais ou exibiam dados inventados.

O projeto foi convertido em uma IA pessoal que roda **inteiramente no computador do usuário**: inferência local via Ollama (ou llama.cpp/LM Studio), banco SQLite com índice lexical e vetorial, base de conhecimento com importação de arquivos e links (com proteção SSRF), respostas com citações verificadas, projetos com contexto isolado, chat com streaming e cancelamento, e um agente local separado que dá acesso somente leitura ao computador, dentro de pastas autorizadas e com auditoria. A interface foi conectada a todas essas funções e modernizada de forma moderada, preservando a identidade visual.

81 + 17	**13/13**	**100%**	**0**
testes backend + agente aprovados	cenários ponta a ponta (sistema real)	recuperação e fidelidade (qwen2.5:7b + bge-m3)	vulnerabilidades conhecidas (npm/pip audit)

Pendências principais: OCR, voz, integração com Google Drive, ações que alteram o computador e multiusuário — todas documentadas com o motivo e o próximo passo (seções 21–25). A integração com a `main` não foi feita: o trabalho está no branch designado, enviado ao remoto, pronto para revisão (seção 20).

3 Objetivo e visão da IA pessoal

Construir um produto próprio que orquestre **modelos abertos locais** com dados e infraestrutura controlados pelo usuário: conhecimento persistente e consultável (RAG), projetos independentes, conversas do dia a dia e integração controlada com o computador — sem depender de serviços pagos de IA.

> Seguindo o documento de referência “Como Criar Sua Própria IA do Zero”, o sistema foi tratado como produto completo (dados, ingestão, API, orquestração, inferência, RAG, persistência, interface, segurança, avaliação). As seções sobre tokenização BPE, Transformers, pré-treinamento, SFT/LoRA, DPO e RLHF foram interpretadas como material didático: **não** foi treinado nem ajustado nenhum modelo (falta objetivo, dados licenciados e hardware); isso consta como evolução futura. O PDF original não estava disponível no ambiente; usou-se a síntese incorporada ao prompt mestre.

4 Arquitetura encontrada inicialmente

Camada	Estado inicial
Frontend	Next.js 16.3.1, React 19, Tailwind 4; uma página; tema claro/escuro; histórico de conversas funcional
Backend	FastAPI; rotas `/api/chat` e `/api/conversations`; `USER_ID` fixo por variável de ambiente
IA	`AsyncOpenAI` com `OPENAI_API_KEY` obrigatória; cada mensagem enviada sem histórico
Banco	Supabase remoto (tabelas `conversations` e `messages`, esquema não versionado)
Implantação	Vercel (frontend) + Render (backend) + Supabase

Testes	nenhum

5 Principais problemas identificados

1. **Dependência obrigatória da OpenAI** e modelo comercial configurado por padrão.
2. **Backend não instalável**: `ModuleNotFoundError: openai` a partir do `requirements.txt`.
3. Sem autenticação; mensagens e conversas sem verificação de dono; título enviado por query string.
4. Chat sem contexto (histórico persistido não era usado) e falhas convertidas em HTTP 500 genérico.
5. Dados inventados na interface (CPU “Intel i7 (24%)”, RAM “16 GB (62%)”, “Online” fixo).
6. Botões sem ação: Pesquisar, Projetos, Aprender, Histórico, Configurações, microfone, voz; anexos/Drive com `alert()`.
7. Bugs de interação (menu de fixadas fechava antes do clique; sugestões desativadas sem conversa).
8. **25 349 arquivos de `node_modules` e 407 de `.next` versionados** (repositório com 234 MB).
9. Next.js 16.3.1 com avisos **críticos** (RCE e SSRF no otimizador de imagens) — relevante para o site publicado.
10. Um backend hospedado (Render) nunca alcançaria Ollama, arquivos ou processos do computador do usuário.
11. Imagens de marca de até 12 MB carregadas na interface; acento índigo destoando do ciano do logotipo.

6 Matriz de requisitos e estado inicial

Versão completa com evidências, prioridades, dependências e critérios de conclusão: `docs/DIAGNOSTICO_INICIAL.md`.

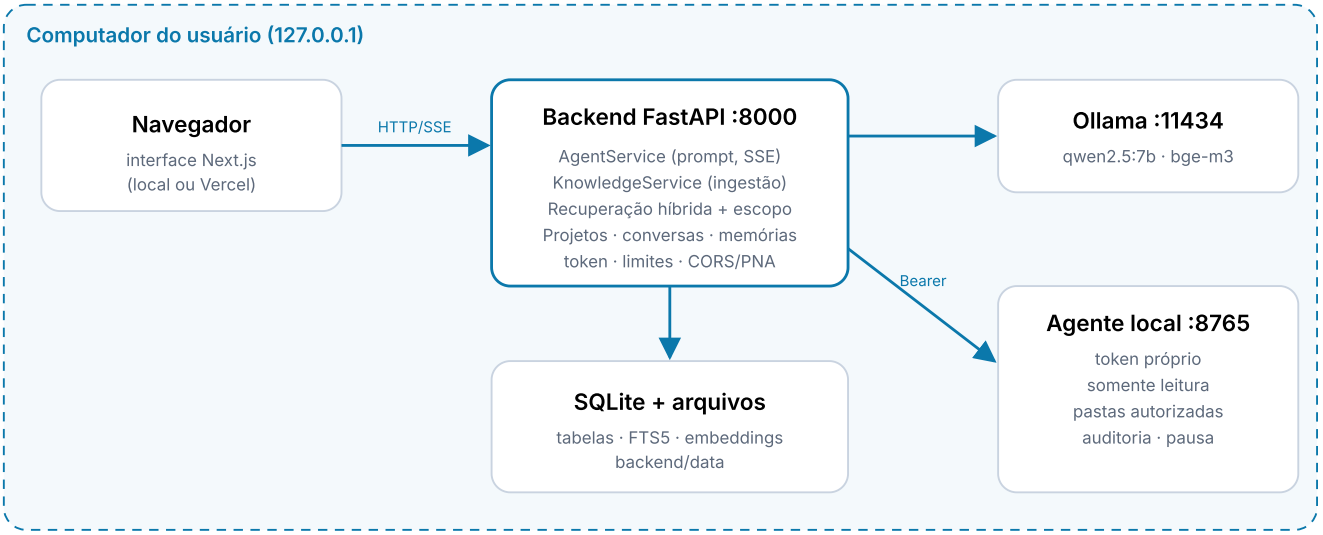
Requisito	Estado inicial	Prioridade	Critério de conclusão
Inferência local sem OpenAI	Ausente	Crítica	Chat responde via Ollama sem chave comercial
Backend instalável	Quebrado	Crítica	Instalação limpa inicia
Estado do runtime	Ausente	Crítica	Interface mostra conectado/desconectado real
Persistência de conversas	Parcial (Supabase)	Crítica	Teste cria/lista/continua conversa
Histórico como contexto	Ausente	Alta	Histórico presente no prompt
Busca/arquivamento	Só interface	Média	Busca por título e conteúdo
Streaming/cancelamento	Ausente	Média	Resposta incremental interrompível
Aprender — arquivos	Só interface	Alta	Formatos indexados e listados
Aprender — links	Ausente	Alta	URL pública indexada; interna recusada
RAG com citações	Ausente	Alta	Resposta cita trechos; teste de fidelidade
Projetos isolados	Só interface	Alta	Projeto A não recupera B
Agente do computador	Só interface (dados falsos)	Alta	Dados reais; acesso fora da pasta negado
Autenticação	Ausente	Crítica	401 sem token quando configurado
Testes automatizados	Ausente	Crítica	Suíte executa e passa
Voz / Google Drive	Só interface	Baixa	Dependem de decisão do usuário

7 Funcionalidades implementadas

Área	Implementação
IA local	Contrato `LLMProvider`; provedores Ollama e OpenAI-compatível local; status, modelos instalados, teste de geração, erros com instrução de configuração
Persistência	SQLite com migrações versionadas, chaves estrangeiras e exclusão em cascata; preferências e log de atividades
Chat	Histórico como contexto, SSE, cancelamento com resposta parcial salva, títulos automáticos, busca, fixar/arquivar/excluir, anexos por conversa, “Lembrar isto”
Aprender	PDF, DOCX, TXT, MD, CSV, XLSX, HTML e links; extração, normalização, chunking com localização, embeddings locais, FTS5, deduplicação, reprocessamento, cancelamento, reindexação, inspeção de trechos, teste de recuperação
RAG	Recuperação híbrida filtrada por escopo no SQL; citações verificadas; aviso de “sem fontes”
Projetos	CRUD, instruções, conhecimento e memórias próprios, atividade, arquivamento, exclusão protegida pelo nome
Agente local	Processo separado com token, 7 ferramentas somente leitura, pastas autorizadas, negação de credenciais, limites, pausa, auditoria, CLI

Interface	Chat, Aprender, Projetos, Histórico, Configurações; estado real da IA/agente; diálogos e notificações acessíveis; responsiva; animações discretas
Dados	Exportação/importação JSON; backup por cópia da pasta de dados
Qualidade	81 testes backend, 17 agente, 13 cenários e2e com axe, avaliação RAG reproduzível

8 Arquitetura final



Componentes, responsabilidades, modelo de dados e fluxos detalhados em `docs/ARQUITETURA.md`. A Vercel só entrega arquivos estáticos: quem conversa com `localhost` é o navegador, e o backend responde ao *preflight* de rede privada apenas para as origens configuradas.

Decisão	Justificativa
Backend na máquina do usuário	Único lugar com acesso ao runtime local, aos arquivos e ao agente; privacidade
SQLite em vez de Supabase	Zero serviços, backup por cópia, dados não saem da máquina; FTS5 nativo
Busca vetorial com NumPy	Suficiente para uso pessoal; trocável por sqlite-vec/Qdrant sem mudar a API
asyncio em vez de Celery/Redis	Estado persistido no banco cobre o volume pessoal sem novos serviços
Agente como processo separado	Privilégios mínimos e isolados; o backend nunca toca o disco do usuário

9 Modelo de IA local utilizado e justificativa

Uso	Modelo	Tamanho	Licença	Justificativa
Chat (padrão)	qwen2.5:7b (Q4)	~4,7 GB	Apache-2.0	Bom português, chamada de ferramentas nativa, roda em 8 GB de RAM; melhor resultado na avaliação
Chat (leve)	qwen2.5:1.5b	~1,0 GB	Apache-2.0	Máquinas modestas; fidelidade 100% na avaliação, mas cita pouco
Embeddings (padrão)	bge-m3	~1,2 GB	MIT	Multilíngue; separou relevante (0,53–0,80) de irrelevante (≤0,31)
Embeddings (leve)	nomic-embed-text	~275 MB	Apache-2.0	Medido: irrelevantes ~0,60–0,65 em português — não separáveis; mantido como opção

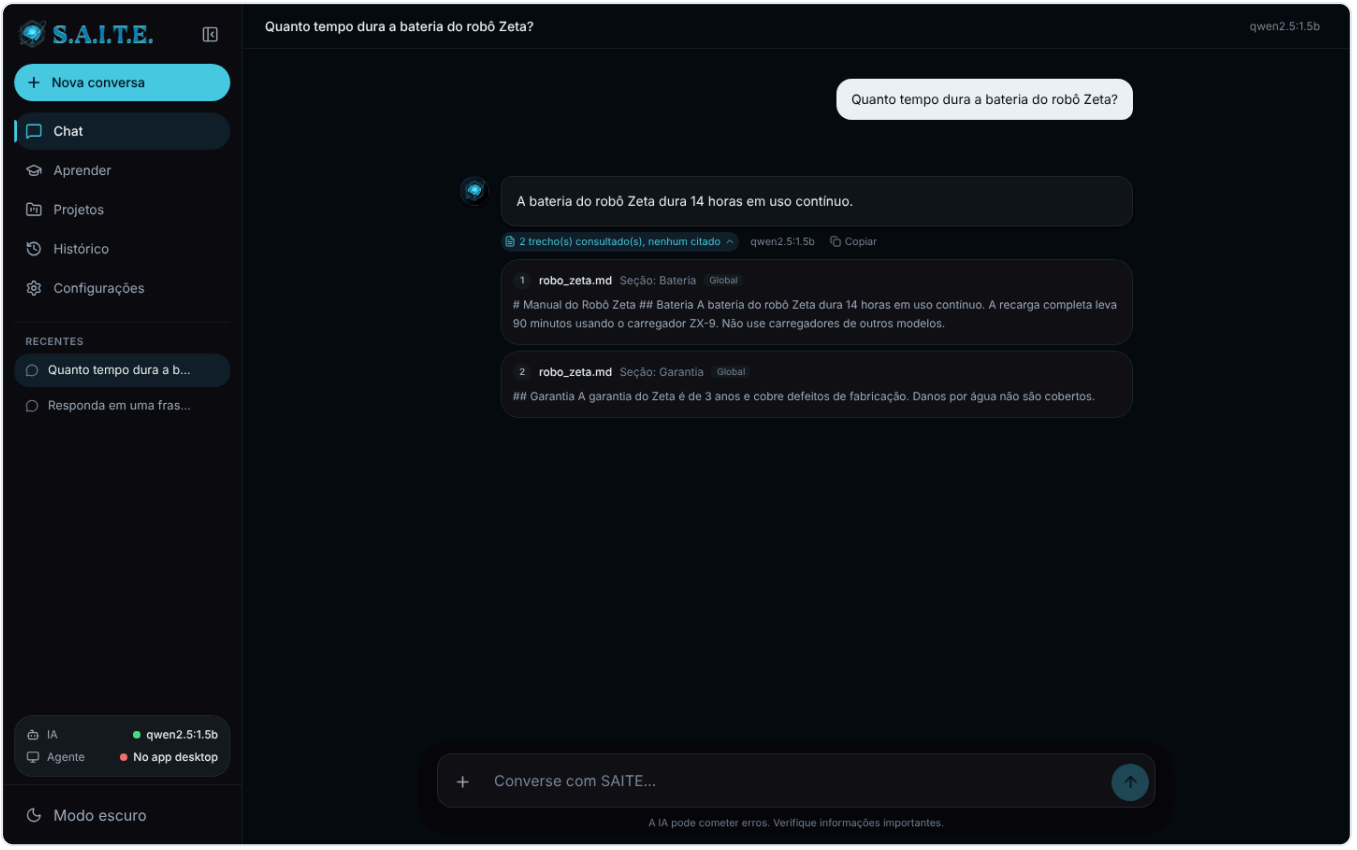
Runtime: **Ollama** (instalação simples em Windows/macOS/Linux, quantização pronta, API de chat/embeddings/ferramentas). llama.cpp e LM Studio funcionam pelo provedor OpenAI-compatível. vLLM não foi adotado (exige GPU de servidor, sem benefício para um único usuário). Nenhum modelo é baixado automaticamente: a interface mostra o comando, o tamanho e a licença. qwen2.5:3b foi evitado por usar licença de pesquisa não comercial.

10 Funcionamento da base de conhecimento

"Aprender" é **aprendizagem por consulta** (RAG), não alteração dos pesos do modelo. Cada documento tem escopo:

Escopo	Onde é criado	Quem enxerga
Global	Aba Aprender	Todas as conversas
Projeto	Detalhe do projeto	Somente conversas do projeto
Conversa	Anexar no chat	Somente aquela conversa (apagado com ela)

Na pergunta: filtro de escopo no SQL → BM25 (FTS5) + cosseno → fusão RRF → limiar absoluto (0,45) e relativo (melhor −0,25) → até 5 trechos entram no prompt, delimitados e marcados como dados não confiáveis → marcadores [n] inválidos são removidos → a interface mostra fontes citadas e consultadas. Memórias (fatos explícitos do usuário) e histórico são mantidos separados da base.



Resposta fundamentada no documento importado, com os trechos consultados (dados reais).

11 Processamento de arquivos e URLs

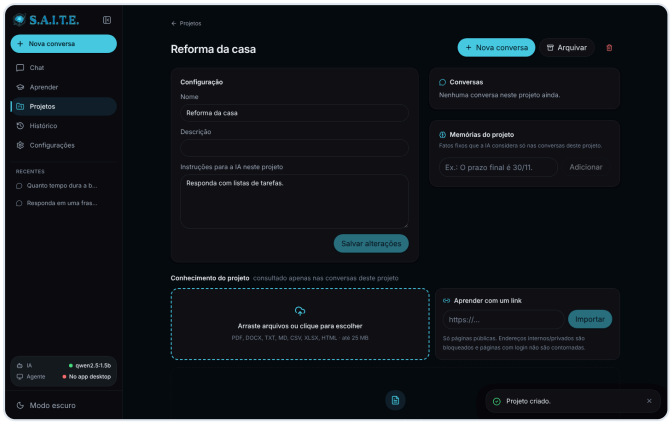
Etapa	Detalhe
Validação	Extensão permitida; assinatura real (%PDF, ZIP); limite 25 MB; ZIP descompactado ≤ 200 MB e razão ≤ 100×; texto sem bytes nulos
Extração	PDF por página; DOCX com títulos e tabelas; XLSX por planilha; CSV em grupos de 50 linhas com cabeçalho; HTML sem script/nav/rodapé; MD por título
Normalização	Unicode NFC, remoção de controles, espaços; títulos sem corpo unidos à seção seguinte
Chunking	~1 600 caracteres (~400 tokens), sobreposição de 200, respeitando parágrafos/frases

Indexação	Embeddings em lotes de 16; gravação atômica de trechos + FTS; sem embeddings → busca só lexical com aviso
Estados	na fila → processando → indexado/erro; cancelamento; reprocessamento incrementa versão; interrupção vira erro recuperável
URLs	Só http/https em 80/443/8080/8443; sem credenciais; todos os IPs resolvidos públicos; conexão no IP validado (anti-rebinding); até 5 redirecionamentos revalidados; 15 s; 5 MB; tipos HTML/TXT/MD/CSV/PDF; 401/403 informados sem contornar

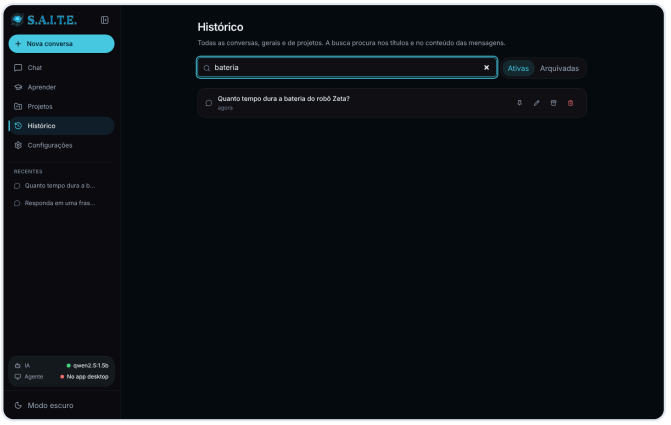
Limitação: PDFs digitalizados geram erro explicativo (OCR não implementado).

12 Funcionamento de projetos e conversas

Conversas gerais usam apenas conhecimento e memórias globais. Uma conversa criada dentro de um projeto (`/?p=<id>`) recebe também as instruções, os documentos e as memórias daquele projeto — e nada de outros projetos (verificado em `test_project_context_does_not_leak`, que inspeciona o prompt enviado ao modelo). Excluir um projeto exige digitar o nome e remove em cascata conversas, mensagens, documentos, trechos, índices, memórias e arquivos; arquivar apenas o oculta.



Detalhe de projeto.



Histórico com busca no conteúdo.

13 Arquitetura e permissões do agente local

Ferramenta	Padrão	Proteções
system_info / disk_usage	ligadas	dados atuais com horário da coleta; montagens auxiliares ocultadas
list_directory / search_files / file_metadata	ligadas	somente dentro das pastas autorizadas (nenhuma por padrão); caminho resolvido seguindo links; sem pastas ocultas; até 200 resultados; tempo limitado
read_text_file	**desligada**	texto até 200 KB; binário recusado
list_processes	**desligada**	sem linha de comando nem usuário

- Sempre negados: `.env`, chaves SSH/PGP/AWS/Azure/Kube, `*.pem`, `*.key`, cofres de senha, perfis de navegador (Chrome/Edge/Firefox/Brave), cofres de credenciais do Windows.
- Escuta só em 127.0.0.1; token gerado no primeiro uso e guardado com permissão 600; 60 chamadas/min; timeout por ferramenta.
- O modelo só recebe as ferramentas quando o usuário ativa “Usar computador”; ferramentas desligadas/inexistentes são recusadas pelo backend e pelo agente.
- Pausa global e revogação imediata (pela interface ou CLI); auditoria JSONL sem conteúdo lido.
- Sem ferramentas de escrita, exclusão, instalação ou execução de comandos (decisão de segurança).

Validação com modelo real: `qwen2.5:7b` chamou `search_files` por iniciativa própria e respondeu com o caminho correto do arquivo.

14 Banco de dados e persistência

SQLite (WAL, chaves estrangeiras, `busy_timeout`) em `backend/data/saite.db`; migrações SQL versionadas em `app/db/migrations`, aplicadas atomicamente na inicialização. Tabelas: `projects`, `conversations`, `messages`, `documents`, `chunks` (+ `chunks_fts` via gatilhos), `memories`, `settings`, `activity_log`, `schema_migrations`. Restrição CHECK garante coerência entre escopo e chaves do documento. Originais em `data/files/<id>/`. Exportação/importação JSON e backup por cópia da pasta.

15 Medidas de segurança

Risco	Medida
Acesso indevido à API	bind em 127.0.0.1; token Bearer opcional (comparação em tempo constante); CORS restrito; PNA só para origens permitidas
SSRF	validação de URL, IPs públicos, conexão fixada no IP, redirecionamentos revalidados (31 testes)
Arquivos maliciosos	assinatura real, zip bomb, limites, nomes higienizados
Injeção SQL/FTS	parâmetros; curingas LIKE escapados; termos FTS citados
Prompt injection / abuso de ferramentas	fontes delimitadas e declaradas não confiáveis; ferramentas só com opt-in, somente leitura, lista fechada
Traversal / credenciais	resolução de caminho + pastas autorizadas + lista de negação
Vazamento	logs e auditoria sem conteúdo; nenhum segredo no frontend/Git; segredos não retornam pela API
Exclusão acidental	confirmação; projeto exige digitar o nome

Dependências	Next.js 16.3.1 → 16.4.3 (corrige RCE/SSRF críticos); npm audit 0; pip-audit 0
Abuso de recursos	limites de taxa, tamanho, contexto e tempo

Riscos residuais: prompt injection não é eliminável; dados em repouso dependem da criptografia do disco; sem token, outros processos locais podem chamar o backend (ver `docs/SEGURANCA.md`).

16 Testes realizados e resultados

Comando	Resultado
`backend: python -m pytest`	81 aprovados
`agent: python -m pytest`	17 aprovados
`frontend: npm run lint` / `npm run typecheck`	0 erros (baseline tinha 1 erro de lint, corrigido)
`frontend: npm run build` (Next 16.4.3)	sucesso — 6 rotas
`frontend: npm run test:e2e` (Playwright + axe, sistema real)	13/13; 0 erros de console
`npm audit` · `pip-audit` (backend e agente)	0 vulnerabilidades

Cobertura por tipo

- **Unitários:** normalização, chunking, extração dos 7 formatos, rejeição de arquivos inválidos, regras SSRF, citações.
- **Integração:** persistência e histórico no prompt, upload→indexação→busca→exclusão, URL ponta a ponta com servidor HTTP local, reprocessamento, fallback lexical, reindexação, isolamento global/projeto/conversa, memórias, exportação/importação, indisponibilidade do modelo, erro no meio do streaming, backend ↔ agente real (processo HTTP), ferramentas recusadas.
- **Segurança:** token, PNA, traversal, links simbólicos, arquivos de credenciais, ferramentas desativadas, pausa, comandos inexistentes, URLs internas, entradas malformadas, limites de taxa.
- **Interface:** estados reais, chat com streaming e persistência, Aprender, RAG com fontes, Projetos, Histórico, Meu PC, opt-in do computador, backend offline, 390 px sem rolagem horizontal, movimento reduzido, WCAG A/AA (axe) em ambos os temas.

Avaliação de RAG (corpus e perguntas fixos em `backend/eval`; 8 respondíveis + 3 sem evidência)

Chat	Embeddings	Recuperação	Fidelidade	Com citação	Precisão citações	Admite ausência	Latência p50
qwen2.5:0.5b	bge-m3	100%	75%	0%	—	33%	1.33 s
qwen2.5:1.5b	bge-m3	100%	100%	13%	100%	67%	2.75 s
qwen2.5:7b	bge-m3	100%	100%	88%	100%	67%	12.77 s

Referência anterior à calibragem: com nomic-embed-text + qwen2.5:0.5b a recuperação foi 75% e todas as perguntas recuperavam 5 trechos, inclusive as sem relação. Latências em CPU de 4 núcleos sem GPU. “Fidelidade” = fatos esperados presentes na resposta (aproximação). Mesmo o 7B atribuiu aos documentos uma resposta sem fonte quando a pergunta induzia (“segundo meus documentos...”); por isso a interface sinaliza deterministicamente respostas sem fontes.

17 Comandos de instalação e execução

```
ollama pull qwen2.5:7b && ollama pull bge-m3

cd backend && python -m venv venv && source venv/bin/activate   # Windows: venv\Scripts\activate
pip install -r requirements.txt && cp .env.example .env
uvicorn app.main:app --host 127.0.0.1 --port 8000

cd frontend && npm install && npm run dev                        # http://localhost:3000
```

```
cd agent && pip install -r requirements.txt                      # opcional
python -m saite_agent roots add "C:\Users\voce\Documents"
python -m saite_agent token        # → SAITE_AGENT_TOKEN no backend/.env
python -m saite_agent serve
```

Guia completo, variáveis de ambiente e solução de problemas: `docs/INSTALACAO.md`.

18 Configuração necessária no computador

- Python 3.11+, Node 20+, Ollama 0.40+; ~6 GB de disco para os modelos padrão; 8 GB de RAM livres (ou GPU com 6 GB+) para o 7B.
- Portas locais 11434, 8000, 3000 e 8765 (todas em 127.0.0.1; não abrir no firewall).
- Para usar o site da Vercel: aceitar a permissão de rede local do navegador; manter a origem em `SAITE_CORS_ORIGINS`.
- Recomendado: `SAITE_API_TOKEN` definido; criptografia de disco do sistema ativa.
- Agente: escolher conscientemente as pastas autorizadas e ligar ferramentas sensíveis só se necessário.

19 Alterações por branch e commit

Branch: `claude/new-session-owvmaf` (designado para esta sessão), a partir de `ec858e81`. 15 commits no padrão Conventional Commits (o commit que adiciona este PDF vem em seguida):

Commit	Mensagem
`e1ef10f4`	chore(repo): stop tracking node_modules and .next build output
`78e63639`	docs(audit): add initial technical diagnosis and requirements matrix
`87582440`	feat(storage): replace Supabase with local SQLite source of truth
`32b76861`	feat(ai): add local inference providers and knowledge ingestion/retrieval
`c1904431`	feat(chat): local agent orchestration, streaming and full API
`96de83b1`	feat(agent): add permission-controlled local computer agent
`0b2004bc`	test(backend): cover ingestion, SSRF, RAG isolation, chat and agent integration
`ecea0adf`	feat(learning): calibrate retrieval and add reproducible RAG evaluation
`a5762b70`	feat(ui): connect every screen to real features and modernize the interface
`6989fdd8`	test(ui): add Playwright + axe end-to-end suite against the real stack
`018a874a`	fix(security): upgrade Next.js 16.3.1 → 16.4.3 to patch critical advisories
`4e8671f2`	docs: rewrite README and add installation, architecture and security guides
`7e451bc2`	docs(report): add technical report PDF and its reproducible generator
`01f7596e`	feat(cloud): move persistence to PostgreSQL/Supabase and add owner login
`ab6775e9`	feat(ui): login screen, cloud backend by default and multi-device e2e

Não foram criadas branches de funcionalidade separadas: as instruções do ambiente determinam um único branch de desenvolvimento e proíbem push para outros branches sem autorização explícita. Os commits foram organizados por unidade coerente (repositório, auditoria, armazenamento, IA/conhecimento, chat/API, agente, testes, calibragem, interface, e2e, segurança, documentação).

20 Situação do merge e do push

Item	Situação
Push do branch `claude/new-session-owvmaf`	**Realizado (confirmado pelo Git)**
Merge na `main`	**Não realizado**
Pull request	**Não criado (aguarda pedido explícito do responsável)**

Motivo: as regras desta sessão permitem push apenas no branch designado. A integração recomendada é um pull request `claude/new-session-owvmaf → main`, com revisão humana. Atenção: a `main` ainda contém `node_modules/.next` versionados; o merge os remove do índice.

Impacto na implantação atual: o backend no Render (OpenAI + Supabase) deixa de ser usado; a interface da Vercel passa a apontar para o backend local de cada usuário. Os dados existentes no Supabase não são migrados automaticamente (o esquema não estava versionado) — se necessário, exporte-os e importe no formato de `/api/data/import`.

21 Funcionalidades pendentes

Pendência	Motivo	Próximo passo
OCR de PDFs digitalizados	Exige dependência nativa (Tesseract) e decisão de idioma/qualidade	Integrar `pytesseract` com detecção de páginas sem texto
Voz (STT/TTS)	Web Speech envia áudio a terceiros; motor local não escolhido	whisper.cpp + Piper, via backend
Google Drive	OAuth de terceiros; decisão de privacidade do usuário	Conector opcional, só leitura, por escopo
Ações que alteram o computador	Risco alto; não exigidas para leitura	Fluxo de confirmação por operação com pré-visualização
Multiusuário/login	Uso pessoal local	Contas locais + tokens por dispositivo
Fila persistente entre reinícios	Reinício vira erro recuperável (suficiente para uso pessoal)	Retomada automática na inicialização
Migração de dados do Supabase	Esquema remoto não versionado	Script de exportação → `/api/data/import`
Execução validada do llama.cpp	Testado só o contrato; Ollama validado com modelos reais	Teste de integração com `llama-server`

22 Limitações conhecidas

- Qualidade depende do modelo: ≤1,5B raramente citam, quase não usam ferramentas e podem afirmar sem base.
- Latência alta em CPU (7B: mediana 12,8 s; até ~27 s no teste).
- Busca vetorial em força bruta: adequada a milhares de trechos, não a milhões.
- Limiar de similaridade calibrado para bge-m3; outros modelos de embeddings exigem ajuste.
- Páginas que dependem de JavaScript não têm o texto extraído; paywalls/logins não são contornados.
- Limites de taxa em memória; um processo, um usuário.
- Testes e2e dependem de Ollama e modelos instalados; não há CI configurado no repositório.

23 Dependências de hardware ou serviços

Configuração	Experiência esperada
CPU 4 núcleos, 16 GB RAM, sem GPU (ambiente de teste)	7B utilizável (10–25 s/resposta); 1,5B ~3 s
GPU com 6–8 GB de VRAM	7B em poucos segundos
8 GB de RAM total	Preferir qwen2.5:1.5b + nomic-embed-text

Serviços externos: nenhum obrigatório. A Vercel é opcional (a interface roda localmente). Importação de links acessa a internet apenas para a URL pedida.

24 Riscos técnicos e recomendações

Risco	Recomendação
Prompt injection via documentos	Autorizar só pastas necessárias; manter ferramentas sensíveis desligadas; revisar a auditoria
Exposição acidental da porta	Definir `SAITE_API_TOKEN`; não usar `--host 0.0.0.0`

Perda de dados	Backup periódico de `backend/data`; exportação JSON
Mudança de modelo de embeddings	Usar *Reindexar* e recalibrar o limiar com `backend/eval`
Avisos futuros de dependências	Rodar `npm audit`/`pip-audit` a cada atualização; adicionar CI
Respostas sem base	Preferir 7B+; observar o aviso “sem fontes”; ampliar o conjunto de avaliação

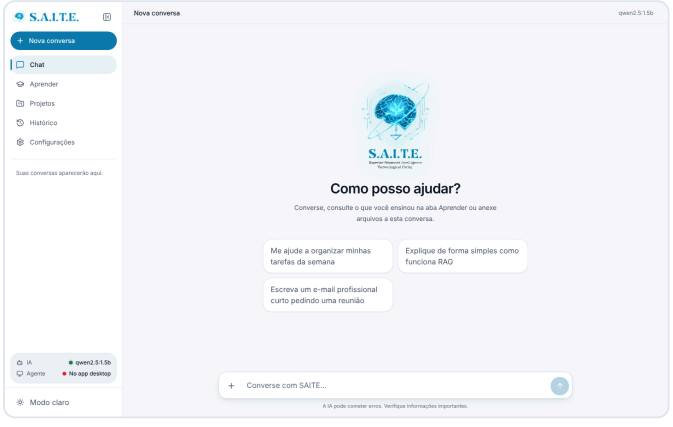
25 Plano de evolução

1. **Curto prazo:** PR para a `main`; CI com pytest, lint, build e audit; OCR; retomada automática de ingestão.
2. **Médio prazo:** reranking local; índice vetorial dedicado (sqlite-vec/Qdrant) para grandes volumes; voz local; ampliação do conjunto de avaliação (mais documentos e perguntas, juiz automático local).
3. **Desktop:** empacotar interface + backend + agente com Tauri (instalação de um clique no Windows), como previsto no README original.
4. **Mobile:** cliente React Native que conversa com o backend da máquina do usuário via túnel autenticado.
5. **Treinamento (condicional):** somente com objetivo, dados licenciados e hardware definidos — LoRA/QLoRA de um modelo aberto, avaliado contra a linha de base RAG desta entrega.

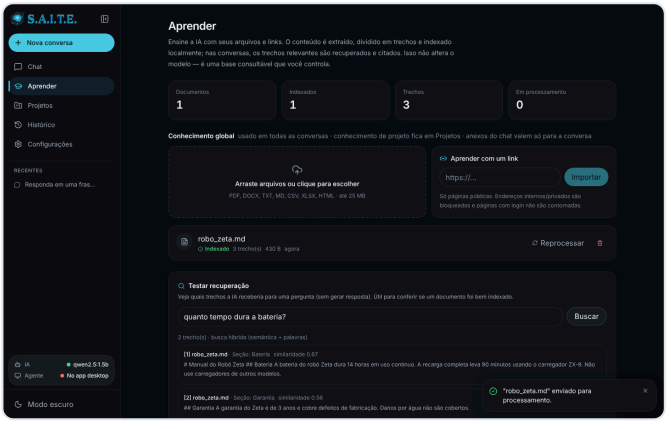
26 Checklist final de entrega

Critério de aceitação	Situação	Evidência
Instalável seguindo a documentação	✓	docs/INSTALACAO.md; instalação limpa usada nos testes
Inferência local sem OpenAI	✓	Ollama + qwen2.5; nenhuma dependência comercial
Indisponibilidade do modelo tratada	✓	testes + e2e “backend offline” + banners
Importar documentos pela aba Aprender	✓	e2e Aprender; 7 formatos testados
Documentos processados e persistidos	✓	SQLite + arquivos; testes de ingestão
Importar links compatíveis	✓	test_import_url_end_to_end
Conteúdo recuperado em consultas	✓	avaliação: recuperação 100%
Respostas apresentam fontes	✓	citações verificadas; e2e RAG
Criar e administrar projetos	✓	testes + e2e Projetos
Projetos com contextos separados	✓	test_project_context_does_not_leak
Conversas persistentes	✓	testes + e2e (recarregamento)
Agente dentro das permissões	✓	17 testes do agente + 5 de integração
Operações sensíveis protegidas	✓	opt-in, ferramentas desligadas, pausa, confirmação de exclusões
Interface conectada às funções reais	✓	remoção de dados falsos; e2e 13/13
Testes e build executados e registrados	✓	seção 16
Documentação atualizada	✓	README, docs/*, agent/README, frontend/README
PDF técnico gerado e validado	✓	este documento (validado após a geração)
Commits profissionais	✓	seção 19
Situação da integração e do push documentada	✓	seção 20 — push no branch designado; merge na main pendente de PR

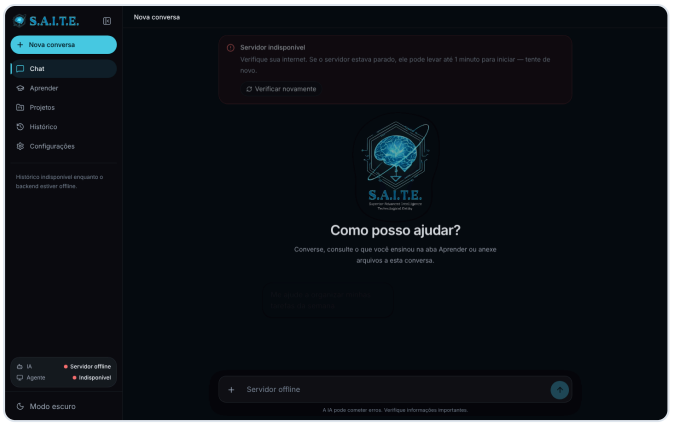
> A entrega não é declarada integralmente concluída em relação a todo o escopo desejado: OCR, voz, Google Drive, ações de escrita no computador, multiusuário e o merge na `main` permanecem pendentes pelos motivos descritos nas seções 20 e 21.



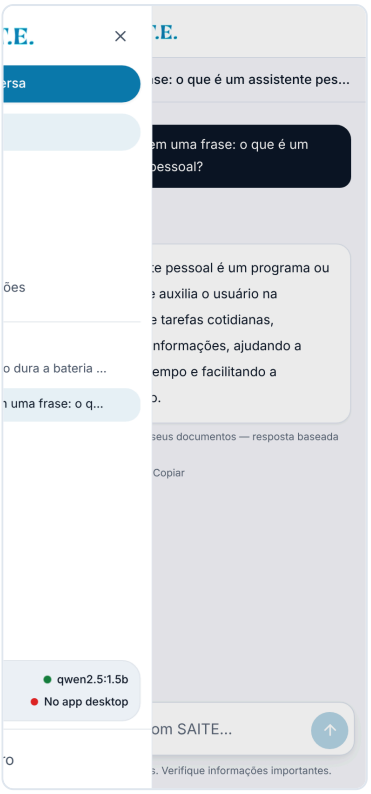
Tema claro.



Aba Aprender.



Estado de erro real (backend offline).



Mobile (390 px).